

Lab Assignment 2

ECE 253 - FA25

November 13, 2025

Problem 1: Geometric Image Transformation

In this assignment, you will implement geometric transformations such as translation, rotation, and shearing. Additionally, you will explore the effect of interpolation when pixel positions in the source image do not map nicely to those in the destination image during a transformation.

Part 1: Implementing Basic and Complex Transformations

Download `geisel.jpg`, and load the image. Apply the following transformations to the image, plot the transformed images along with the original image.

1. Translation

Write a function `apply_translation(image, tx, ty)` that applies a translation to an image. Use the following matrix for translation:

$$\begin{bmatrix} 1 & 0 & tx \\ 0 & 1 & ty \end{bmatrix}$$

2. Rotation

Write a function `apply_rotation(image,  $\theta$ )` that rotates an image by an angle θ (in degrees). Use the following matrix for rotation:

$$\begin{bmatrix} \cos(\theta) & -\sin(\theta) & 0 \\ \sin(\theta) & \cos(\theta) & 0 \end{bmatrix}$$

3. Shearing

Write a function `apply_shearing(image, shx, shy)` to apply a shearing transformation. Use the following matrix for shearing:

$$\begin{bmatrix} 1 & shx & 0 \\ shy & 1 & 0 \end{bmatrix}$$

4. Scaling

Write a function `apply_scaling(image, sx, sy)` that scales an image. The scaling matrix is:

$$\begin{bmatrix} sx & 0 & 0 \\ 0 & sy & 0 \end{bmatrix}$$

Part 2: Transformation with Interpolation

In this part, you will scale an image (geisel.jpg) to a 1024x1024 resolution. Since the pixel positions in the transformed image may not map directly to integer positions in the source image, interpolation is required to estimate pixel values.

1. Scaling with Integer Coordinates (Easy Case)

First, apply a transformation that scales the image to 1024x1024.

- Write a function `nearest_neighbor_interpolation(image, new_size)` that performs the scaling operation and fills in the pixel values using Nearest-neighbor interpolation.
- Write a function `bilinear_interpolation(image, new_size)` that performs the scaling operation and fills in the pixel values using Bilinear interpolation.
- Do not use python libraries for interpolation purposes.
- Plot the results for each interpolation method and compare them.

2. Scaling with Non-Integer Coordinates (Challenging Case)

Now, apply the scaling transformation where the scaling factor results in non-integer transformed pixel coordinates (e.g., scaling by 3.15). As discussed in class, this will make the destination image pixel positions non-integer values, requiring inverse mapping and interpolation.

- Write a function `apply_scaling_non_integer_coords(source_image, scaling_factor)` that:
 1. Takes non-integer transformed coordinates in the destination image and rounds them to the nearest integer coordinates.
 2. Inversely maps these rounded coordinates back to the source image, resulting in non-integer source coordinates.
 3. Interpolates the pixel values in the source image using:
 - Nearest-neighbor interpolation.
 - Bilinear interpolation.
 4. Use the interpolated pixel values to fill in the pixels corresponding to the transformed coordinates in the destination image.
- Plot the results and compare the interpolation methods.

Problem 2: Image Decimation

1. Read the image `spatial_freq.jpg` provided.

2. **Design a Filter:**

- Apply a simple averaging filter to smooth the image before downsampling. Use the following filter:

$$H = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

3. **Perform Decimation:**

- Downsample the signal or image by a factor of 4 and 16.
- Visualize the results and compare them to the original signal or image.

4. **Analyze the Results:**

- Compare the original signal/image with the decimated versions.
- Observe any artifacts or changes in the signal/image. Provide an explanation for these changes.

Problem 3: Interpolation with a Custom Filter

Objective: Investigate how a custom filter affects interpolation when performing upsampling and down-sampling.

1. Read the image `spatial_freq.jpg` provided.

2. **Design a Filter:**

- Apply the following filter:

$$H = \frac{1}{25} \begin{bmatrix} 1 & 2 & 3 & 2 & 1 \\ 2 & 3 & 4 & 3 & 2 \\ 3 & 4 & 5 & 4 & 3 \\ 2 & 3 & 4 & 3 & 2 \\ 1 & 2 & 3 & 2 & 1 \end{bmatrix}$$

3. **Perform Up- and Down-Sampling:**

- First, upsample the signal or image by a factor of 2, apply the custom filter, and then downsample it back to the original resolution.
- Visualize the final result and compare it with the original signal or image.

4. **Analyze the Results:**

- Observe any differences between the original and the processed signal/image.
- Explain any distortions, artifacts, or unexpected changes. Provide a hypothesis for why these changes occurred based on your filter design.